Week 5 Tutorial

Exercises 1–8 — determine the output displayed

1. 20

2. 18

3. Atlantic, Pacific, Antarctic

4. 6
   8
   10
   12

5. 5
   6
   7

6. 10
   9
   8
   7
   6
   5
   4
   3
   2
   1

7. 3 20

8. 4

Exercises 9–10 — rewrite the code

9. Rewrite using a **for** loop:

```
for num in range(1, 10, 2):
print(num)
```

10. Rewrite using a **while** loop:

```
total = int(input("Enter a number: "))
count = 1
while count < 3:
    num = int(input("Enter a number: "))
    total = total + num
    count += 1
print(total)
```

Exercises 11–14 — determine the output displayed

11. This sentence contains five words.

    This sentence contains six different words.

12. Hearth, Damith

13. Largest Number: 8

    Length: 4

    Total: 17

14. When in the course of human events

Exercises 15–17 — identify all errors

15. Error - Index out of range, it asks to print the 5th element. But the code only has 4 elements.

    Fix - print(virtues[3])

16. Error - When using the “join()” method, all elements need to be strings, but list has integers

    Fix - print(" ".join(str(item) for item in list1))

17. Errors -

    1. Missing colon after 'while True'
    2. 'num = 1' resets counter each iteration (should be outside loop)
    3. No condition to finish the loop

    Fix -

```
num = 0
while True:
    num = num + 1
    print(num)
    if num >= 5:
        break
```

Laboratory

1)
```
celsius = 10
print("Celsius | Fahrenheit")
print("-" * 20)

while celsius <= 30:
    fahrenheit = (9/5 * celsius) + 32
    print(f"{celsius:7} | {fahrenheit:10}")
    celsius += 5
```

2)
```
phrase = input("Enter a phrase: ")
vowel_count = 0
vowels = "aeiouAEIOU"

for char in phrase:
    if char in vowels:
        vowel_count += 1

print(f"The phrase contains {vowel_count} vowels.")
```

Case Study

Part A — Input loop (while) & list of purchases

```
# Starter
```

```
movies = {
    "Dune": 12.5,
    "Barbie": 11.0,
    "Oppenheimer": 13.0,
    "Spirited Away": 10.0
}

# TODO: use a while loop that continues until the user enters 'done'
while True:
    # 1) ask for movie title (case sensitive is fine)
    title = input("Enter movie title (or 'done' to finish): ")

    if title.lower() == 'done':
        break

    # 2) if title not in movies, print the available keys and continue
    if title not in movies:
        print(f"Movie '{title}' not found. Available movies:", ", ".join(movies.keys()))
        continue

    # 3) ask for quantity (int)
    qty = int(input("Enter quantity: "))

    # 4) append (title, qty, movies[title]) to purchases
    purchases.append((title, qty, movies[title]))
```

```
    # 5) optional: track running subtotal
    running_subtotal = 0
    for t, q, p in purchases:
        running_subtotal += q * p
    print(f"Running subtotal: ${running_subtotal:.2f}")
```

Part B — Receipt (for loop over list)

```
# TODO: iterate over purchases and compute line totals
print("\n" + "=" * 50)
print("RECEIPT")
print("=" * 50)

grand_total = 0
print(f"{'Movie':20} {'Qty':5} {'Price':8} {'Total':10}")
print("-" * 50)

for title, qty, price_each in purchases:
    # line_total = ...
    line_total = qty * price_each

    # print(...)
    print(f"{title:20} {qty:5} ${price_each:7.2f} ${line_total:9.2f}")
    grand_total += line_total
```

```
# print("Total:", ...)
print("-" * 50)
print(f"{'GRAND TOTAL:':30} ${grand_total:9.2f}")
```

Part C — Sales Summary (dictionaries)

```
tickets_by_movie = {}
revenue_by_movie = {}

# TODO: for each (title, qty, price_each) in purchases:
for title, qty, price_each in purchases:
    #   update tickets_by_movie[title]
    if title in tickets_by_movie:
        tickets_by_movie[title] += qty
    else:
        tickets_by_movie[title] = qty

    #   update revenue_by_movie[title]
    revenue = qty * price_each
    if title in revenue_by_movie:
        revenue_by_movie[title] += revenue
    else:
        revenue_by_movie[title] = revenue
```

```
# TODO: print each movie with tickets and revenue
print("\n" + "=" * 50)
print("SALES SUMMARY")
print("=" * 50)
print(f"{'Movie':20} {'Tickets':10} {'Revenue':12}")
print("-" * 50)

for title in movies.keys():
    tickets = tickets_by_movie.get(title, 0)
    revenue = revenue_by_movie.get(title, 0)
    print(f"{title:20} {tickets:10} ${revenue:11.2f}")
```

Part D — Discounts & validation (challenge)

```
# def apply_group_discount(qty, price_each):
def apply_group_discount(qty, price_each):
    """Apply 10% discount if buying 4+ tickets"""
    line_total = qty * price_each
    if qty >= 4:
        discount = line_total * 0.10
        line_total -= discount
        print(f"  Group discount (10%): -${discount:.2f}")
    return line_total
```

 # def apply_member_discount(total, is_member):

```
def apply_member_discount(total, is_member):
    """Apply 5% member discount to total"""
    if is_member:
        discount = total * 0.05
        total -= discount
        print(f"Member discount (5%): -${discount:.2f}")
    return total

print("\n" + "=" * 50)
print("PART D — DISCOUNTS & VALIDATION")
print("=" * 50)

if purchases:
    discounted_total = 0
    original_total = 0

    for title, qty, price_each in purchases:
        original_line = qty * price_each
        original_total += original_line

        print(f"\n{title}: {qty} tickets at ${price_each:.2f} each")

        # Use try/except to guard against non-numeric quantities
        try:
```

```
            # Convert qty to int to ensure it's numeric
            qty = int(qty)
            discounted_line = apply_group_discount(qty, price_each)
            discounted_total += discounted_line
            print(f"  Line total after discount: ${discounted_line:.2f}")
        except ValueError:
            print(f"  Error: Invalid quantity '{qty}' - skipping discount")
            discounted_total += original_line

    print(f"\nOriginal total: ${original_total:.2f}")
    print(f"Total after group discounts: ${discounted_total:.2f}")

    # Ask if the customer has a member code
    member_response = input("\nDoes the customer have a member code? (yes/no): ")
    is_member = member_response.lower() == 'yes'

    final_total = apply_member_discount(discounted_total, is_member)
    print(f"\nFINAL TOTAL after all discounts: ${final_total:.2f}")
    print(f"Total savings: ${original_total - final_total:.2f}")
else:
    print("No purchases to process.")
```

Part E — Analytics (stretch)

```
print("\n" + "=" * 50)
print("PART E — ANALYTICS")
print("=" * 50)

if purchases and tickets_by_movie:
    # Example sketches (fill in the logic)

    # Find the top-selling movie by tickets
    top_title = None
    top_qty = -1

    for title, qty in tickets_by_movie.items():
        if qty > top_qty:
            top_title, top_qty = title, qty

    print(f"\nTop-selling movie by tickets: {top_title} ({top_qty} tickets)")

    # sorted_by_rev = sorted(revenue_by_movie.items(), key=lambda kv: kv[1], reverse=True)
    # print(sorted_by_rev)

    # Sort titles by revenue
    sorted_by_rev = sorted(revenue_by_movie.items(), key=lambda kv: kv[1], reverse=True)

    print("\nMovies ranked by revenue:")
    for i, (title, revenue) in enumerate(sorted_by_rev, 1):
        print(f"  {i}. {title}: ${revenue:.2f}")
```

```
# Compute the average tickets per purchase using the purchases list
total_tickets = sum(tickets_by_movie.values())
num_purchases = len(purchases)
avg_tickets = total_tickets / num_purchases if num_purchases > 0 else 0

print(f"\nAverage tickets per purchase: {avg_tickets:.2f}")
print(f"Total tickets sold: {total_tickets}")
print(f"Total number of purchases: {num_purchases}")
```